BULUT DEPOLAMA ve YÖNETİLEN VERİ HİZMETLERİ

İÇİNDEKİLER

BULUT DEPOLAMA ve YÖNETİLEN VERİ HİZMETLERİ 1
Tablolar Listesi 2
Şekiller Listesi 2
1. Dersin Amacı ve Öğrenme Çıktıları 3
1.1. Dersin amacı 3
1.2. Öğrenme Çıktıları 3
2. Bulut Depolamanın Temelleri 3
2.1. Verinin Depolanması ve Erişim Gereksinimi 3
2.2. Kapasite ve Performans Ölçütleri 4
3. Bulutta Depolama Modelleri 5
3.1. Blok Depolama 5
3.2. Dosya Depolama 6
3.3. Nesne Depolama 7
3.4. Depolama Modellerinin Karşılaştırılması ve Seçimi 8
4. Veri Yaşam Döngüsü ve Koruma 9
4.1. Depolama Katmanları ve Yaşam Döngüsü Kuralları 9
4.2. Nesne Sürümleme ve Veri Çoğaltma 10
4.3. Yedekleme ve Geri Yükleme 11
4.4. Veri Koruma Yöntemlerinin Birlikte Değerlendirilmesi 13
5. Yönetilen Veri Hizmetleri 14
5.1. Yönetilen Veritabanı Kavramı 14
5.2. Yönetilen İlişkisel Veritabanları 15
5.3. Bulutta Yönetilen NoSQL Veritabanları 16
5.4. Önbellek ve Veri Gölü 17
5.5. Veri Hizmeti Seçim Ölçütleri 17
6. Kavramların Bulut Sağlayıcılarındaki Karşılıkları 19
6.1. Depolama modelleri 19
6.2. Yönetilen veri hizmetleri 19
7. Gerçek Dünya Kullanım Senaryosu: Üniversite Tez Teslim Sistemi 19
7.1. Üniversite Tez Teslim Sisteminin Gereksinimleri 19

7.2. Verinin Yerleştirilmesi ve Erişim Akışı 20
7.3. Yaşam Döngüsü ve Geri Yükleme Kararları 20
8. Bölüm Özeti 21
Kaynakça 22
Hafta 4 – Çalışma Soruları 25

Tablolar Listesi

Tablo 1. Blok, dosya ve nesne depolamanın uygulamaya sunduğu erişim yapısına göre karşılaştırılması. [1], [2], [4] 8
Tablo 2. Veri yaşam döngüsü ve koruma yöntemlerinin amaçlarına göre karşılaştırılması. [6], [9]–[13] 13
Tablo 3. Veri türü ve erişim gereksinimine göre yönetilen veri hizmeti seçimi. 18
Tablo 4. Üç depolama modeline ilişkin hizmet adı örnekleri. [25]–[27] 19
Tablo 5. Yönetilen veri hizmetlerine ilişkin örnek adlar. [28]–[30] 19

Şekiller Listesi

Şekil 1. Tez dosyasının nesne olarak, teslim bilgilerinin ise sorgulanabilir kayıtlar olarak saklanması. Şekildeki iki veri kolu aynı işleme aittir; uygulama bunların arasındaki ilişkiyi korumalıdır. 7
Şekil 2. Hatalı nesne silme ve hatalı veritabanı güncellemesinde geri dönüş yolları. Nesne için önceki sürümün korunmuş, veritabanı için de uygun yedek veya belirli zamana geri dönüş seçeneğinin mevcut olduğu varsayılmıştır [9], [11]–[13]. 12
Şekil 3. Tez dosyası ile teslim kaydının ayrı hizmetlerde tutulması ve yetkili erişim akışı. 20

1. Dersin Amacı ve Öğrenme Çıktıları

1.1. Dersin amacı

Bu haftanın amacı, öğrencinin bir bulut uygulamasında farklı veri türlerinin neden farklı depolama ve veri hizmetleri gerektirdiğini kavramasıdır. Öğrenci; blok, dosya ve nesne depolamayı erişim biçimlerine göre karşılaştıracak, yönetilen veritabanı ve önbellek gibi hizmetlerin mimarideki yerini belirleyecek, veri yaşam döngüsü ile temel veri koruma kararlarını gerekçelendirecektir.

1.2. Öğrenme Çıktıları

Bu bölümün sonunda öğrenci:

1. Blok, dosya ve nesne depolamayı erişim biçimi, performans ve kullanım alanı açısından karşılaştırarak uygun modeli seçer.
2. Veri türü ve sorgulama gereksinimine göre yönetilen ilişkisel veritabanı, NoSQL veritabanı, önbellek ve veri gölünün kullanımını gerekçelendirir.
3. Erişim sıklığı ve saklama gereksinimine uygun bir veri yaşam döngüsü önerir.
4. Sürümleme, çoğaltma ve yedeklemenin amaçlarını ayırt ederek yanlış silme veya hatalı güncelleme için bir geri yükleme yolu belirler.
5. Basit bir uygulama senaryosunda depolama ve veri hizmetlerini birlikte gösteren bir mimari tasarlar.

2. Bulut Depolamanın Temelleri

Bir bulut uygulaması farklı özelliklerde veriler üretir ve kullanır: işletim sistemi dosyaları, kullanıcı belgeleri, işlem kayıtları veya analiz için biriktirilen veriler. Bunların hepsini aynı depolama hizmetinde tutmak teknik olarak mümkün olsa bile, uygulamanın veriye erişim biçimi ve veriden beklediği davranış farklıdır. Bu nedenle depolama tasarımında ilk karar, kapasite veya sağlayıcı ürün adı değil, verinin uygulama tarafından nasıl kullanılacağıdır [1], [2].

Bu bölümde önce erişim gereksinimi ele alınacaktır. Kapasite ve performans ölçütleri 2.2'de, erişim sıklığı ile maliyet ilişkisi 2.3'te incelenecektir. Blok, dosya ve nesne depolamanın çalışma biçimleri ise 3. bölümde karşılaştırılacaktır.

2.1. Verinin Depolanması ve Erişim Gereksinimi

Bir veri için depolama hizmeti seçmeden önce şu soru sorulur: Uygulama bu veriyi hangi birim üzerinden okuyup değiştirecek? Bir sanal makine kendisine bağlı bir disk üzerinde işletim sistemi ve uygulama dosyalarını çalıştırmak isteyebilir. Birden fazla sunucu aynı dosya yolunu kullanmak isteyebilir. Bir web uygulaması ise kullanıcının yüklediği fotoğrafı veya PDF belgesini bir bütün olarak kaydedip daha sonra çağırabilir. Bu üç durumda saklanan içerik dijital veri olsa da erişim gereksinimleri aynı değildir [1], [2].

Erişim biriminden sonra okuma ve yazma örüntüsü belirlenir. Veri sık mı okunuyor? Küçük parçalar hâlinde ve sık mı değiştiriliyor? Genellikle bir kez yazılıp uzun süre mi saklanıyor? Aynı veriye eşzamanlı olarak kaç uygulama erişiyor? Örneğin öğrenci notları sık sorgulanabilen ve güncellenebilen kayıtlardır; teslim edilmiş bir tez PDF'si ise çoğunlukla bütün bir belge olarak saklanır. Bu fark, yalnız depolama modelini değil, veritabanı gibi bir veri hizmetine ihtiyaç olup olmadığını da etkiler [1]–[3].

Son olarak verinin ne kadar süreyle ve hangi koşullarda erişilebilir kalması gerektiği tanımlanır. Günlük kullanılan bir dosyanın erişim beklentisiyle yıllarca saklanacak bir arşivin beklentisi farklıdır. Yanlışlıkla silinen veya değiştirilen verinin geri getirilebilmesi gerekiyorsa bu gereksinim de baştan kaydedilir. Böylece depolama seçimi, yalnız "veri nereye konacak?" sorusuyla sınırlı kalmaz; verinin kullanımını ve korunmasını birlikte ele alır [1].

Kısa örnek: Üniversite tez teslim sisteminde tez PDF'si, öğrencinin teslim tarihi ve jüri onay durumu aynı iş sürecine aittir. Ancak PDF'nin içeriğine erişim ile teslim ve onay kayıtlarını sorgulama ihtiyacı farklıdır. Mimaride bu veriler birbiriyle ilişkilendirilir; her birinin nerede tutulacağı ise kendi erişim gereksinimine göre belirlenir. Bu örneğin hizmet seçimi 3. ve 5. bölümlerde yapılacaktır.

2.2. Kapasite ve Performans Ölçütleri

Depolama hizmetinin yeterliliği yalnız saklayabildiği veri miktarıyla belirlenmez. Aynı büyüklükteki iki veri kümesi, farklı okuma ve yazma davranışları nedeniyle farklı performans gerektirebilir. Bu nedenle kapasite ve performans ayrı ölçülür [1], [2].

Kapasite, saklanan verinin miktarıdır; genellikle gigabayt (GB) veya terabayt (TB) ile ifade edilir. İhtiyaç hesaplanırken bugünkü veri miktarına ek olarak verinin büyüme hızı, tutulacak eski sürümler ve yedekler düşünülür. Örneğin her biri 10 MB olan 100.000 belge yaklaşık 1 TB ham içerik oluşturur. Bu değer, aynı belgelerin ek kopyalarını ve üstverilerini kapsayan nihai kapasite hesabı değildir.

Gecikme (latency), bir okuma veya yazma isteğinin sonuçlanması için geçen süredir. Küçük veri parçalarına art arda erişen bir uygulamada gecikme önem kazanır. Örneğin bir veritabanı, işlem sırasında çok sayıda küçük okuma yapıyorsa her isteğin bekleme süresi uygulamanın yanıtını etkileyebilir.

Aktarım hızı (throughput), belirli sürede taşınabilen veri miktarıdır; örneğin MB/s ile ifade edilir. Büyük bir dosyanın okunması veya çok sayıda dosyanın toplu aktarılması sırasında öne çıkar. Yüksek aktarım hızı, her tekil isteğin düşük gecikmeyle tamamlanacağı anlamına gelmez.

Saniyedeki giriş/çıkış işlemi sayısı (input/output operations per second, IOPS), depolama sisteminin bir saniyede gerçekleştirebildiği okuma ve yazma işlemlerinin sayısını belirtir. Özellikle çok sayıda küçük ve rastgele erişim içeren iş yüklerini değerlendirirken kullanılır.

IOPS değeri tek başına yorumlanmamalıdır: işlem başına veri boyutu ve okuma–yazma oranı değiştiğinde aynı IOPS değeri farklı aktarım hızlarına karşılık gelebilir [1].

Örneğin bir üniversitenin tez PDF arşivi büyük kapasite ve belgeyi okurken yeterli aktarım hızı gerektirebilir. Öğrenci kayıt veritabanı ise daha az yer kaplamasına rağmen çok sayıda kısa okuma ve yazma işlemi nedeniyle gecikme ve IOPS bakımından daha duyarlı olabilir. Seçim yapılırken bu ölçütler, uygulamanın gerçek erişim örüntüsüyle birlikte değerlendirilir [1], [2].

Hatırlanacak ilişki: Kapasite "ne kadar veri?", gecikme "bir istek ne kadar bekler?", aktarım hızı "birim zamanda ne kadar veri taşınır?", IOPS ise "birim zamanda kaç işlem yapılır?" sorularını yanıtlar.

3. Bulutta Depolama Modelleri

Bulutta depolama hizmetleri, uygulamaya veriyi hangi yapıda sunduklarına göre üç temel modelde incelenir: blok, dosya ve nesne depolama. Bu ayrım, verinin fiziksel olarak hangi disk türünde bulunduğunu değil, uygulamanın veriye nasıl eriştiğini anlatır[1], [2].

Blok depolama bir sisteme disk benzeri alan sunar. Dosya depolama, dosya ve dizinlerden oluşan bir yapıya erişim sağlar. Nesne depolamada ise uygulama, veriyi bir anahtar aracılığıyla nesne olarak kaydeder ve çağırır. Bir uygulama bu modellerden yalnız birini kullanmak zorunda değildir: işletim sistemi diski, sunucular arasında paylaşılan dosyalar ve kullanıcı belgeleri aynı mimaride farklı modellerde tutulabilir.

Bu bölümde önce her modelin çalışma biçimi ve uygun kullanım alanı açıklanacak; ardından modeller birlikte karşılaştırılacaktır. Hizmet seçiminde temel ölçüt, verinin adı veya dosya uzantısı değil, uygulamanın gerektirdiği erişim biçimidir [1], [2].

3.1. Blok Depolama

Block storage (blok depolama), veriyi sabit veya değişken büyüklükte adreslenebilir bloklar hâlinde sunar. Bir sanal makine bu alanı disk olarak görür. İşletim sistemi diski bölümlendirebilir, üzerinde bir dosya sistemi oluşturabilir ve dosyaları bu dosya sistemi aracılığıyla okuyup yazabilir. Depolama hizmeti blokları sağlar; dosya ve dizin düzenini ise işletim sistemi yönetir [1], [2].

Blok depolamanın en anlaşılır kullanım alanı sanal makinenin işletim sistemi diskidir. İşletim sisteminin ve kurulu uygulamaların dosyaları bu diskte bulunabilir. Bir diğer örnek, veritabanı yazılımının çalıştığı sunucuya bağlanan veri diskidir. Bu durumda uygulama diskin bloklarına doğrudan sorgu göndermez; veritabanı yazılımı ve işletim sistemi depolama alanını kullanır.

Blok depolama, küçük parçalar üzerinde sık okuma ve yazma yapan iş yüklerinde değerlendirilebilir. Ancak "blok depolama her durumda daha hızlıdır" genellemesi doğru

değildir. Sonuç; seçilen hizmetin özelliklerine, sağlanan performans düzeyine ve uygulamanın erişim örüntüsüne bağlıdır. Bu nedenle 2.2'deki gecikme, aktarım hızı ve IOPS ölçütleri birlikte incelenir [1], [2].

Bulutta bir blok birimin sanal makineye bağlanması, birden fazla makinenin aynı dosyaları güvenle paylaşabildiği anlamına gelmez. Ortak erişim gerekiyorsa dosya sistemi ve eşzamanlı yazma desteği ayrıca değerlendirilir; çoğu temel senaryoda bu gereksinim için 3.2'deki dosya depolama modeli daha doğrudan bir başlangıç noktasıdır.

Depolama alan ağı yaklaşımında sunucuya ağ üzerinden bir mantıksal disk sunulması, blok depolama fikrini anlamaya yardımcı olur. Bulut kullanıcısı ise çoğunlukla fiziksel diskleri ve bağlantı donanımını yapılandırmak yerine, gereksinimine uygun bir mantıksal birim oluşturup sanal makineye bağlar. Yönetim sınırının bu şekilde değişmesi, blok erişimin temel niteliğini değiştirmez.

Örnek: Tez teslim uygulamasının çalıştığı sanal makinenin işletim sistemi için blok depolama uygundur. Öğrencilerin yüklediği binlerce PDF'yi yalnızca bu makinenin diskinde biriktirme kararı ise ayrıca değerlendirilmelidir: uygulama birden fazla sunucuya yayıldığında dosyalara erişim ve saklama düzeni değişecektir.

3.2. Dosya Depolama

File storage (dosya depolama), veriyi dosyalar ve dizinlerden oluşan bir yapı içinde sunar. Uygulama bir belgeye `raporlar/2026/sonuc.pdf` gibi bir dosya yolu üzerinden erişir. Dosya sistemi; dosya adlarını, dizin yapısını ve dosyalara erişimi yönetir. Bulutta bu yapı, ağ üzerinden erişilebilen bir dosya paylaşımı olarak sağlanabilir [2], [4].

Dosya depolamanın belirgin özelliği, birden fazla istemcinin aynı dosya alanına erişebilmesidir. Örneğin üç uygulama sunucusu aynı rapor şablonlarını kullanıyorsa, her sunucuda ayrı kopya tutmak yerine ortak bir dosya paylaşımına bağlanabilir. Bu yaklaşım, uygulamanın dosya yolu ve dizin yapısı beklediği durumlarda özellikle yararlıdır.

Ağ üzerinden dosya erişiminde yaygın protokoller arasında Network File System (NFS; ağ dosya sistemi) ve Server Message Block (SMB; sunucu ileti bloğu) bulunur. Hangi protokolün kullanılacağı, işletim sistemi ve uygulamanın erişim gereksinimine bağlıdır. Bu haftada protokollerin paket düzeyindeki işleyişinden çok, uygulamaya paylaşılan bir dosya sistemi sunmaları önemlidir [4].

Ortak dosya alanı, eşzamanlı kullanım sorunlarını kendiliğinden çözmez. İki uygulama aynı dosyayı aynı anda değiştirecekse erişim ve güncelleme düzeni uygulama tarafından düşünülmelidir. Ayrıca dosya paylaşımının performansı; dosya sayısı, dosya boyutu, eşzamanlı kullanıcı sayısı ve okuma–yazma örüntüsüne göre değerlendirilir. Yalnız "paylaşımlı erişim gerekiyor" demek, her iş yükü için aynı dosya hizmetinin uygun olduğunu göstermez [1], [2].

Dosya depolama ile blok depolama arasındaki temel fark, uygulamaya sunulan yapıdır. Blok depolamada sisteme disk benzeri bir alan verilir ve dosya sistemini kullanan sistem yönetir. Dosya depolamada ise uygulama, erişime hazır bir dosya ve dizin yapısıyla çalışır. Nesne depolamadan farkı 3.3'te ele alınacaktır.

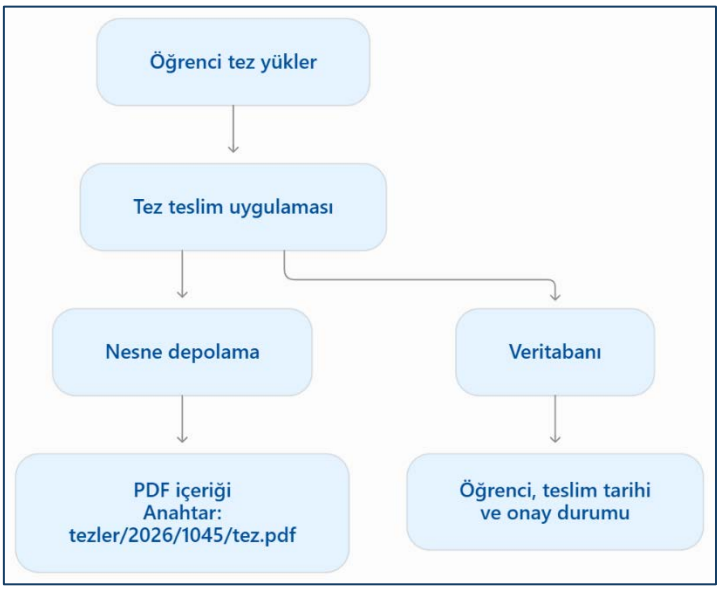
Örnek: Bir bölümün kullandığı üç uygulama sunucusunun aynı belge şablonlarına dosya yolu üzerinden erişmesi gerekiyorsa dosya depolama uygun bir adaydır. Buna karşılık yalnızca yüklenip daha sonra indirilecek çok sayıdaki öğrenci belgesinin depolanmasında, uygulamanın gerçekten ortak dosya yolu gerektirip gerektirmediği ayrıca sorgulanmalıdır.

3.3. Nesne Depolama

Object storage (nesne depolama), veriyi bir nesne (object) olarak saklar. Nesne; içerik, onu tanımlayan bir anahtar (key) ve üstveriden (metadata) oluşur. Nesneler genellikle bir kova (bucket) içinde düzenlenir. Uygulama, nesneyi bir disk bloğu veya işletim sistemindeki dosya yolu üzerinden değil, depolama hizmetinin sunduğu arayüz üzerinden anahtarını belirterek kaydeder ve okur [2], [5].

Örneğin bir tez teslim uygulaması PDF içeriğini `tezler/2026/1045/tez.pdf` anahtarıyla saklayabilir. Anahtardaki `/` karakterleri kullanıcıya dizin görünümü verebilir; ancak bu gösterim, nesne deposunun geleneksel bir dosya sistemi gibi çalıştığı anlamına gelmez. Uygulama tez sahibini, teslim tarihini ve onay durumunu ayrıca bir veritabanında tutabilir. PDF nesnesi ile veritabanı kaydı bir tanımlayıcı aracılığıyla ilişkilendirilir.

Şekil 1, aynı teslim işleminin iki farklı veri gereksinimi doğurduğunu gösterir:



Şekil 1. Tez dosyasının nesne olarak, teslim bilgilerinin ise sorgulanabilir kayıtlar olarak saklanması. Şekildeki iki veri kolu aynı işleme aittir; uygulama bunların arasındaki ilişkiyi korumalıdır.

Nesne depolama; kullanıcıların yüklediği belge ve görseller, video dosyaları, arşivler ve analiz için toplanan veri dosyaları gibi içerikler için uygundur. Özellikle uygulama içeriği

bir bütün olarak yükleyip daha sonra anahtarıyla çağırıyorsa güçlü bir adaydır. Buna karşılık bir sanal makinenin açılış diski gibi işletim sisteminin blok düzeyinde erişim beklediği bir gereksinim için doğrudan aynı işlevi görmez [1], [2].

Bir nesnenin içindeki küçük bir bölümü sürekli değiştirmeyi gerektiren uygulama da dikkatle değerlendirilmelidir. Nesne deposu, uygulamaya genel amaçlı bir disk veya paylaşımlı dosya sistemi olarak sunulmaz; güncelleme yöntemi ve maliyeti hizmetin özelliklerine göre incelenir. Nesne saklama ile nesne üzerinde karmaşık sorgular çalıştırma gereksinimi de ayrıdır: öğrenci numarasına göre tezleri bulmak gibi sorgular için uygun bir veri modeli veya ek indeks gerekebilir [2], [5].

Nesne depolama hizmetlerinde saklama sınıfı, yaşam döngüsü, sürümleme ve çoğaltma seçenekleri bulunabilir. Bunlar nesne modelinin erişim biçimini değiştirmez; verinin ne kadar süreyle, hangi maliyet ve koruma koşullarıyla saklanacağını belirler. Bu seçenekler 4. bölümde ele alınacaktır [5].

3.4. Depolama Modellerinin Karşılaştırılması ve Seçimi

Blok, dosya ve nesne depolama arasındaki seçim, verinin uzantısından veya büyüklüğünden önce uygulamanın beklediği erişim biçimine dayanır. Aynı PDF belgesi, ortak bir dizindeki dosya olarak da nesne deposundaki bir nesne olarak da saklanabilir. Belirleyici soru, uygulamanın bu belgeye dosya yolu üzerinden mi yoksa bir nesne anahtarı ve hizmet arayüzü üzerinden mi erişeceğidir [1], [2].

Tablo 1 üç modeli temel seçim ölçütleriyle karşılaştırır.

Tablo 1. Blok, dosya ve nesne depolamanın uygulamaya sunduğu erişim yapısına göre karşılaştırılması. [1], [2], [4]

Ölçüt	Blok depolama	Dosya depolama	Nesne depolama
Sisteme sunulan yapı	Disk benzeri mantıksal birim	Dosya ve dizinlerden oluşan paylaşım	Anahtar ve üstveriyle tanımlanan nesne
Tipik erişim	İşletim sistemi ve dosya sistemi üzerinden	Dosya yolu üzerinden	Hizmet arayüzü ve nesne anahtarı üzerinden
Uygun gereksinim	Sanal makine diski; disk erişimi bekleyen yazılım	Birden çok sistemin kullandığı ortak dosya alanı	Belge, görsel, video ve arşiv gibi bütün hâlde saklanan içerik
Dosya sistemini kim sunar?	Bağlanan sistem oluşturur ve yönetir	Depolama hizmeti erişime hazır dosya yapısı sunar	Geleneksel dosya sistemi erişimi temel model değildir
Seçimde kritik soru	“Uygulamaya disk gerekiyor mu?”	“Ortak dosya yolu gerekiyor mu?”	“İçerik nesne olarak kaydedilip çağrılabilir mi?”

Tablodaki kullanım alanları kesin kurallar değil, seçim için başlangıç noktalarıdır. Örneğin veritabanı yazılımı bir sanal makinede çalışıyorsa depolama altyapısında blok birim

kullanılabilir. Uygulama yönetilen bir veritabanı hizmeti kullanıyor ise kendi veritabanı diskini oluşturup yönetmesi gerekmez. Benzer biçimde birden fazla uygulama sunucusunun aynı belgeleri kullanması, mutlaka ortak dosya paylaşımı gerektirmez; uygulamanın nesne deposuna erişecek biçimde tasarlanıp tasarlanmadığına bakılır.

Seçimi sistematik yapmak için şu sıra izlenebilir:

1. Erişim birimini belirle: Disk blokları mı, dosya yolu mu, nesne anahtarı mı gerekli?
2. Erişen sistemleri belirle: Tek bir sanal makine mi, birden fazla sunucu mu, yoksa uygulama üzerinden çok sayıda kullanıcı mı erişecek?
3. İş yükünü ölç: Küçük ve sık işlemler mi, büyük ve seyrek aktarımlar mı baskın?
4. Koruma ve saklama gereksinimini yaz: Veri ne kadar tutulacak ve yanlış değişiklikten nasıl geri dönülecek?
5. Uygun modeller arasından seçim yap: Performans ve toplam maliyeti bu aşamada karşılaştır.

Örneğin tez teslim sisteminde sanal makinenin açılış diski için blok depolama, birkaç sunucunun aynı şablon dizinine erişmesi için dosya depolama, yüklenen tez PDF'leri için nesne depolama seçilebilir. Öğrenci, tez ve onay kayıtlarının nasıl tutulacağı ise depolama modelleriyle birlikte veri hizmeti seçimi gerektirir; bu konu 5. bölümde ele alınacaktır. Böylece Şekil 1'de gösterilen PDF içeriği ile teslim kaydının neden farklı yerlerde tutulduğu da açıklığa kavuşur.

4. Veri Yaşam Döngüsü ve Koruma

Depolama modeli seçildikten sonra verinin zaman içinde nasıl yönetileceği belirlenir. Yeni oluşturulan bir belge sık kullanılırken birkaç yıl sonra yalnız yasal veya kurumsal saklama amacıyla tutulabilir. Ayrıca bir dosyanın yanlışlıkla silinmesi, hatalı bir içerikle değiştirilmesi veya depolama hizmetinde kesinti yaşanması farklı koruma önlemleri gerektirir.

Bu bölümde önce verinin erişim sıklığına göre depolama sınıfının ve saklama süresinin nasıl yönetildiği açıklanacaktır. Ardından sürümleme, çoğaltma, yedekleme ve geri yükleme ele alınacaktır. Buradaki amaç, öğrencinin temel bir veri koruma kararı verebilmesidir.

4.1. Depolama Katmanları ve Yaşam Döngüsü Kuralları

Storage tier (depolama katmanı), verinin erişim sıklığı ve beklenen geri alma süresine uygun saklama seçeneğini ifade eder. Genel olarak sık kullanılan veri için hızlı erişim, seyrek kullanılan veri için daha düşük saklama maliyeti, uzun süre saklanan arşiv için ise farklı geri alma koşulları ön plana çıkar. Katmanın adı ve kesin davranışı sağlayıcıya göre değiştiğinden seçim yalnız "sıcak" veya "soğuk" etiketine bakılarak yapılmaz [2], [6]–[8].

Bir tez teslim sistemini düşünelim. Değerlendirmesi süren tez PDF'lerine öğrenciler ve jüri üyeleri sık erişebilir. Değerlendirmesi biten tezler daha seyrek okunur; önceki yılların tezleri ise uzun süre saklanmasına rağmen nadiren açılır. Aynı dosya türü için erişim sıklığı zamanla değiştiğinden, başlangıçta uygun olan saklama seçeneği ilerleyen dönemde yeniden değerlendirilebilir.

Lifecycle policy (yaşam döngüsü politikası), belirlenen koşulları sağlayan veriye uygulanacak işlemleri tanımlar. Nesne depolamada bu işlemler, örneğin bir nesneyi başka bir depolama sınıfına geçirmek veya saklama süresi sonunda silmek olabilir. Bir kural nesnenin yaşı, konumu ya da başka uygun koşullarına bağlanabilir. AWS S3, Azure Blob Storage ve Google Cloud Storage bu amaçla kural tabanlı yaşam döngüsü yönetimi sunar [6]–[8].

Örnek bir politika şöyle ifade edilebilir:

"Tez değerlendirmesi tamamlanan PDF'ler belirlenen süre boyunca erişime hazır tutulur. Daha sonra, beklenen kullanım ve geri alma koşulları uygunsa seyrek erişimli sınıfa geçirilir. Kurumun saklama süresi dolmadan silinmez."

Bu, doğrudan uygulanacak hazır bir kural değil, tasarım örneğidir. Gerçek politikada "değerlendirme tamamlandı" bilgisinin nasıl temsil edildiği, saklama süresinin nereden başladığı ve silme yetkisi açıkça tanımlanmalıdır. Yaşam döngüsü işlemlerinin uygulama zamanı da her hizmette tam olarak kuralın koşulunun sağlandığı ana eşit olmayabilir; uygulama bu işlemin anında gerçekleşeceğini varsaymamalıdır [8].

Katman değiştirirken yalnız GB başına saklama ücretine bakmak yeterli değildir. Daha seyrek erişimli bir sınıfta okuma veya geri alma bedeli, asgari saklama süresi, erken silme ücreti ve geri alma süresi bulunabilir. Veriye tekrar sık erişilecekse düşük saklama bedeli toplam maliyet açısından avantaj sağlamayabilir. Bu nedenle politika, erişim örüntüsü + geri alma beklentisi + toplam maliyet + saklama yükümlülüğü birlikte değerlendirilerek belirlenir [6]–[8].

Yaşam döngüsü kuralı, yanlışlıkla değiştirilen bir dosyanın eski içeriğini kendiliğinden geri getirmez. Bu ihtiyaç için sürümleme; farklı konumda kopya bulundurma ve bağımsız geri dönüş ihtiyaçları için de sonraki alt başlıklarda açıklanan yöntemler değerlendirilir.

4.2. Nesne Sürümleme ve Veri Çoğaltma

Versioning (sürümleme), bir nesnenin üzerine yeni içerik yazıldığında önceki hâlinin de saklanmasını sağlayan veri koruma yöntemidir. Örneğin `tez.pdf` dosyasının düzeltilmiş hâli yüklendiğinde eski sürüm korunuyorsa, yanlış dosya yüklenmesi durumunda önceki içerik bulunabilir. Sürümleme hizmette etkinleştirilir; hangi eski sürümlerin ne kadar süre tutulacağı ayrıca planlanır [9].

Sürümleme, özellikle yanlışlıkla üzerine yazma veya silme olaylarında yararlıdır. Bununla birlikte sınırsız sayıda eski sürümün tutulması kullanılan kapasiteyi ve maliyeti artırır. Eski

sürümler için yaşam döngüsü kuralı tanımlanırken kurumun saklama gereksinimi ile geri dönüş ihtiyacı birlikte değerlendirilir. Ayrıca sürümleme, bütün depolama alanının veya hesabın silinmesi gibi her olay için tek başına yeterli koruma olarak kabul edilmemelidir [9].

Replication (çoğaltma), verinin bir kaynaktan başka bir depolama konumuna kopyalanmasıdır. Kopya aynı bölgede veya farklı bölgede tutulabilir; kullanılabilen seçenekler hizmete bağlıdır. Amaç, örneğin veriye başka bir konumdan erişmek ya da belirli bir konumun kullanılamadığı durumda bir kopyayı hazır bulundurmaktır [10]. Çoğaltmanın yapılandırılması, verinin her koşulda anında ve eksiksiz olarak ikinci konumda bulunacağını varsayma hakkı vermez; kopyalamanın durumu ve gecikmesi izlenmelidir.

Bu iki yöntem farklı soruları yanıtlar:

- Sürümleme: “Bu nesnenin önceki içeriğine dönebilir miyim?”
- Çoğaltma: “Bu nesnenin başka bir konumda kopyası var mı?”

Örneğin uygulama saat 14.00’te doğru bir PDF’nin üzerine hatalı PDF yazarsa, çoğaltma yeni hatalı içeriği de diğer konuma taşıyabilir. Önceki sürüm korunmuşsa eski PDF’ye dönüş mümkün olabilir. Tersine, bir bölgede erişim kesintisi yaşandığında yalnız eski sürümlerin aynı bölgede bulunması, başka konumdan erişim gereksinimini karşılamaz. Bu nedenle sürümleme ve çoğaltma birbirini tamamlayabilir; biri diğerinin yerine geçmez [9], [10].

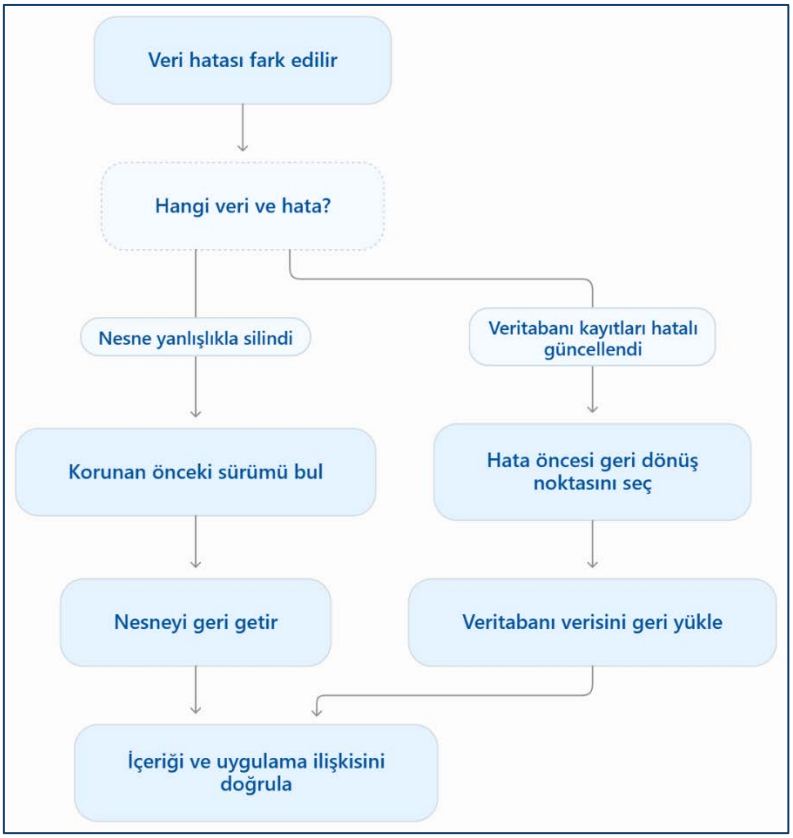
Örnek politika: Tez dosyalarında önceki sürümleri belirli bir süre tutmak ve seçilen dosyaları ikinci bir konuma çoğaltmak mümkündür. Politika yazılırken hangi dosyaların çoğaltılacağı, eski sürümlerin de kopyalanıp kopyalanmayacağı, kopyaların ne zaman silineceği ve geri dönüşün nasıl yapılacağı açıkça belirtilmelidir. Yanlış değişikliklerden bağımsız bir geri dönüş kopyası oluşturma gereksinimi ise 4.3’teki yedekleme kapsamında değerlendirilir.

4.3. Yedekleme ve Geri Yükleme

Backup (yedekleme), verinin daha sonra geri getirilebilmesi için belirli bir durumunun korunmasıdır. Restore (geri yükleme) ise korunan durumdan yeniden kullanılabilir veri elde etme işlemidir. Yedek alınması tek başına yeterli değildir: hangi verinin korunduğu, yedeğin ne zaman alındığı ve geri yüklemenin çalışıp çalışmadığı bilinmelidir [11].

Yedekleme gereksinimi veri türüne göre değişir. Bir tez PDF’si için önceki nesne sürümünü geri almak yeterli olabilir. Öğrenci, tez ve onay kayıtlarından oluşan bir veritabanında ise belirli bir andaki tutarlı veri durumuna dönmek gerekir. Yönetilen veritabanı hizmetleri otomatik yedekleme veya point-in-time recovery (belirli bir zamana geri dönüş, PITR) sunabilir; kullanılabilirlik, saklama süresi ve etkinleştirme koşulları hizmete göre kontrol edilir [11]–[13].

Örneğin saat 14.00'te yapılan hatalı bir işlem 200 öğrencinin onay durumunu değiştirmiş olsun. Geri dönüşte amaç, rastgele seçilmiş en son kopyayı yüklemek değildir. Hatanın gerçekleştiği zaman belirlenir, uygun geri dönüş noktası seçilir, kayıtlar geri yüklenir ve sonuç doğrulanır. Hizmetin geri yüklemeyi mevcut veritabanının üzerine mi yoksa yeni bir veritabanına mı yaptığı da önceden öğrenilmelidir; örneğin Google Cloud SQL'in PITR işlemi yeni bir örnek oluşturur [13].



Şekil 2. Hatalı nesne silme ve hatalı veritabanı güncellemesinde geri dönüş yolları. Nesne için önceki sürümün korunmuş, veritabanı için de uygun yedek veya belirli zamana geri dönüş seçeneğinin mevcut olduğu varsayılmıştır [9], [11]–[13].

Şekil 2, iki farklı hata için geri dönüş kaynağının neden farklı seçildiğini gösterir. Yanlışlıkla silinen bir nesnede korunmuş önceki sürüm aranırken, hatalı güncellenen veritabanında hata öncesindeki uygun geri dönüş noktası belirlenir. Her iki durumda da geri getirilen verinin doğruluğu ve uygulamadaki ilişkileri kontrol edilir.

Temel bir yedekleme planı şu soruları yanıtlamalıdır:

1. Ne korunacak? Dosya içeriği, veritabanı kayıtları ve bunlar arasındaki ilişki.
2. Hangi geçmişe dönülebilecek? Kullanılabilir yedek veya geri dönüş noktaları.
3. Yedekler ne kadar tutulacak? Kurumsal saklama ve maliyet gereksinimleri.
4. Kim geri yükleyebilecek? Yetkili kişiler ve uygulanacak işlem.

5. Sonuç nasıl doğrulanacak? Dosyanın açılması veya kayıtların bütünlüğünün kontrolü.

Tez teslim sistemi örneğinde yalnız PDF dosyalarını geri getirmek yeterli olmayabilir. Veritabanındaki teslim kaydı PDF nesnesini işaret ediyorsa, geri yükleme sonrasında bu ilişkinin hâlâ doğru olup olmadığı kontrol edilir. Bu, veri korumanın tek tek kopyalardan çok uygulamanın yeniden doğru çalışmasıyla ilgili olduğunu gösterir.

Bu haftada öğrenciden ayrıntılı bir felaket kurtarma planı hazırlaması beklenmez. Temel beklenti, yanlış silme veya hatalı güncelleme için uygun geri dönüş kaynağını seçmesi ve geri yükleme sonucunu doğrulamasıdır.

4.4. Veri Koruma Yöntemlerinin Birlikte Değerlendirilmesi

Yaşam döngüsü yönetimi, sürümleme, çoğaltma ve yedekleme aynı veriye uygulanabilir; ancak farklı soruları yanıtlar. Bu yöntemlerden birinin etkin olması, diğerlerinin sağladığı korumanın da mevcut olduğu anlamına gelmez. Tasarım yapılırken önce korunulacak olay tanımlanır, ardından uygun yöntem seçilir.

Tablo 2. Veri yaşam döngüsü ve koruma yöntemlerinin amaçlarına göre karşılaştırılması. [6], [9]–[13]

Yöntem	Temel amacı	Örnek kullanım	Tek başına karşılamadığı ihtiyaç
Yaşam döngüsü yönetimi	Veriyi belirlenen koşullarda taşımak veya silmek	Eski tezleri seyrek erişimli sınıfa geçirmek	Hatalı değişiklikten geri dönüş
Sürümleme	Nesnenin önceki hâllerini korumak	Üzerine yanlış PDF yüklenen tezin eski sürümünü bulmak	Başka konumdan erişim garantisi
Çoğaltma	Verinin başka bir konumda kopyasını bulundurmak	Belgelerin ikinci konumda da bulunması	Hatalı güncellemeden önceki doğru duruma dönüş
Yedekleme	Belirli bir veri durumundan geri dönüş sağlamak	Hatalı güncellenen kayıtları uygun noktadan geri yüklemek	Geri yükleme yapılmadan uygulamanın çalışmayı sürdürmesi

Örneğin tez teslim sisteminde PDF'ler için sürümleme, önceki yılların belgeleri için yaşam döngüsü kuralı, konum kaynaklı erişim ihtiyacı için çoğaltma ve öğrenci–tez–onay kayıtları için geri yüklenebilir veritabanı yedeği birlikte değerlendirilebilir. Bunların tümünü her veriye aynı biçimde uygulamak yerine, verinin önemine ve karşılaşılabilecek hata türüne göre karar verilir.

Son kontrol sorusu şudur: "Bir hata gerçekleştiğinde doğru veriyi gerçekten geri getirebilir ve uygulamanın o veriyle doğru çalıştığını gösterebilir miyiz?" Koruma ayarlarının varlığı kadar, geri dönüş yolunun bilinmesi ve sonucunun doğrulanması da gereklidir.

5. Yönetilen Veri Hizmetleri

Önceki bölümlerde blok, dosya ve nesne depolamanın veriye erişim biçimini inceledik. Bir uygulamanın gereksinimi ise kimi zaman dosyayı saklamaktan daha fazlasıdır: "Bu öğrencinin onaylanmış tezlerini bul", "Son teslim tarihine göre sırala" veya "Bir onay işlemi sırasında ilgili kayıtları birlikte güncelle" gibi işlemler gerekir. Bu durumda verinin yapısını ve sorgulanmasını yöneten bir veri hizmeti seçilir [3].

Bu bölüm; yönetilen veritabanı kavramını, ilişkisel ve NoSQL veritabanlarını, önbelleği ve veri gölünü temel kullanım amaçlarıyla ele alır. Amaç, her hizmetin ayrıntılı yönetimini öğrenmek değil, bir uygulamadaki veriye hangi hizmetin neden uygun olduğunu açıklayabilmektir.

5.1. Yönetilen Veritabanı Kavramı

Managed database (yönetilen veritabanı), veritabanı hizmetinin kurulumu ve işletimiyle ilgili bazı altyapı görevlerinin bulut sağlayıcısı tarafından yürütüldüğü hizmet modelidir. Kullanıcı bir sanal makine oluşturup veritabanı yazılımının bütün işletim süreçlerini kendi yönetmek yerine, veritabanı hizmetini kullanır. Sağlayıcının üstlendiği görevler ürüne göre değişmekle birlikte altyapı sağlama, yazılım bakımı ve yedekleme işlemlerinin bir bölümünü içerebilir [14]–[16].

Bu durum, uygulama ekibinin veritabanı hakkında karar vermeyeceği anlamına gelmez. Ekip hâlâ şu konulardan sorumludur:

- Verinin hangi yapıda tutulacağını ve tablolar veya diğer veri yapıları arasındaki ilişkileri belirlemek.
- Uygulamanın yapacağı sorguları ve veri güncellemelerini tasarlamak.
- Kimlerin hangi verilere erişebileceğini belirlemek.
- Sunulan kapasite ve performans seçeneklerinden gereksinime uygun olanı seçmek.
- Yedekleme ayarlarını ve ihtiyaç duyulan geri dönüş yolunu doğrulamak.

Örneğin tez teslim sisteminde sağlayıcı, yönetilen veritabanının yazılım bakımını yapabilir. Ancak Öğrenci, Tez ve Onay kayıtlarının nasıl ilişkilendirileceğine, yanlış bir onayın nasıl düzeltileceğine ve hangi kullanıcının onay verebileceğine uygulama ekibi karar verir. Yönetilen hizmet, altyapı işletiminin bir kısmını devralır; veri modelinin doğruluğunu kendiliğinden sağlamaz.

Yönetilen veritabanı ile blok depolama da karıştırılmamalıdır. Veritabanının altında sonuçta bir depolama altyapısı bulunur; fakat uygulama yönetilen hizmete disk blokları üzerinden değil, veritabanının sunduğu sorgu ve veri erişim arayüzü üzerinden bağlanır.

Böylece depolama biriminin seçimi ile veritabanı hizmetinin seçimi farklı mimari kararlar olarak görülür.

Yönetilen hizmetler arasında da farklar vardır. Bir hizmet ilişkisel tablo ve SQL sorgularını desteklerken diğeri belge veya anahtar–değer odaklı olabilir. Bu nedenle "yönetilen" sözcüğü verinin nasıl modelleneceğini değil, hizmetin nasıl işletildiğini belirtir. Veri modeline göre seçim 5.2 ve 5.3'te yapılacaktır [14]–[16].

5.2. Yönetilen İlişkisel Veritabanları

Relational database (ilişkisel veritabanı), veriyi tablolar hâlinde düzenler. Her tablo belirli bir varlık veya olaya ilişkin kayıtları tutar; tablolar arasındaki bağlantılar anahtarlar aracılığıyla kurulur. Structured Query Language (yapılandırılmış sorgu dili, SQL), bu verileri sorgulamak ve değiştirmek için kullanılır [3].

Tez teslim sisteminde Öğrenci, Tez ve Onay tabloları düşünülebilir. Tez kaydı ilgili öğrenciyi, Onay kaydı da ilgili tezi gösterir. Böyle bir düzen, "Bir öğrencinin onaylanmış tezleri hangileridir?" gibi bir soruyu tablolar arasındaki ilişkileri kullanarak yanıtlamayı sağlar. PDF'nin içeriği nesne deposunda bulunabilir; veritabanında ise onu tanımlayan nesne anahtarı ve teslim bilgileri tutulur.

İlişkisel veritabanı seçiminde önemli bir gerekçe veri bütünlüğüdür. Örneğin var olmayan bir teze onay kaydı eklenmemesi veya bir işlemde birden fazla kaydın birlikte doğru güncellenmesi istenebilir. Transaction (işlem), ilişkili veri değişikliklerinin tanımlanmış kurallara göre birlikte yürütülmesini sağlar. Bu ders kapsamında işlem yönetiminin bütün teknik ayrıntılarının bilinmesi gerekmez; ilişkili kayıtların tutarlı kalmasının neden önemli olduğunun kavranması yeterlidir.

Bulutta yönetilen ilişkisel veritabanı kullanıldığında uygulama, hizmetin sunduğu veritabanı bağlantısı ve SQL arayüzüyle çalışır. Sağlayıcı altyapı ve bakım işlerinin bir bölümünü üstlenirken uygulama ekibi tablo tasarımını, sorguları, erişim izinlerini ve ihtiyaç duyduğu geri yükleme ayarlarını değerlendirir [14]–[16]. Yönetilen hizmet kullanılması, kötü tasarlanmış bir sorguyu veya hatalı veri ilişkisini kendiliğinden düzeltmez.

Bu model özellikle şu gereksinimlerde güçlü bir adaydır:

- Veriler arasında açık ve korunması gereken ilişkiler varsa.
- Birden fazla tabloyu birleştiren sorgular gerekiyorsa.
- İlişkili kayıtların birlikte doğru güncellenmesi önemliyse.
- Veri yapısı ve bütünlük kuralları belirlenebiliyorsa.

Örneğin "tez PDF'lerini saklamak" tek başına ilişkisel veritabanı seçme gerekçesi değildir. Buna karşılık hangi öğrencinin hangi tezi ne zaman teslim ettiği ve kimlerin onayladığı

ilişkisel kayıtlar olarak modellenebilir. Böylece dosya içeriği ile işlem kayıtları, kendi kullanım biçimlerine uygun hizmetlerde tutulur.

5.3. Bulutta Yönetilen NoSQL Veritabanları

NoSQL (Not Only SQL; yalnız SQL ile sınırlı olmayan), veriyi ilişkisel tablolar dışında farklı modellerle düzenleyen veritabanlarını kapsayan genel addır. Tek bir veri modeli veya bütün NoSQL hizmetlerinde ortak olan bir sorgu dili değildir. Anahtar–değer, belge, geniş sütun ve grafik modelleri bu grupta yer alır [17], [18].

Anahtar–değer (key–value) modelinde veriye bir anahtarla erişilir. Belge (document) modelinde ise bir kayda ait alanlar birlikte tutulur; belgeler genellikle koleksiyonlarda düzenlenir. Örneğin bir öğrencinin bildirim tercihleri, öğrenci kimliğiyle bulunan bir kayıt veya belge olarak modellenebilir. Bu kayıtların hangi alanları taşıyacağı ve nasıl doğrulanacağı yine uygulama tasarımının parçasıdır; NoSQL kullanılması "veri modeline gerek yok" anlamına gelmez [17]–[19].

Bulutta bu modeller yönetilen NoSQL hizmetleri olarak sunulur. Sağlayıcı veritabanı altyapısının kurulması, bakımı ve ölçeklenmesine ilişkin işletim yükünün önemli bir bölümünü üstlenir. Uygulama ekibi ise verinin nasıl düzenleneceğine, en sık hangi sorguların yapılacağına, kayıtların hangi anahtarla bulunacağına ve gerekli erişim ile geri yükleme ayarlarına karar verir [20]–[22]. Dolayısıyla hizmetin yönetilen olması, veri modelleme kararını ortadan kaldırmaz.

Bu ilişki üç sağlayıcının hizmetlerinde görülebilir:

- AWS Amazon DynamoDB, yönetilen bir NoSQL hizmetidir; anahtar–değer ve belge türündeki veriler için kullanılabilir. Tasarımda kaydın anahtarı ve beklenen erişim yolları önem taşır [20].
- Microsoft Azure Cosmos DB for NoSQL, belge odaklı veriler için yönetilen bir hizmettir. Verinin bölümlere dağıtılmasında kullanılan partition key (bölümleme anahtarı), erişim ve kapasite davranışını etkileyen bir tasarım kararıdır [21].
- Google Cloud Firestore, veriyi koleksiyonlardaki belgeler olarak düzenleyen yönetilen bir NoSQL hizmetidir. Uygulama, belgelerin yapısını ve bunlara nasıl erişeceğini tasarlar [19], [22].

Bu örnekler bire bir eşdeğer ürünler olarak okunmamalıdır. Üçünde de yönetilen NoSQL yaklaşımı vardır; ancak veri modeli, sorgu imkânları, ölçekleme ayarları, işlem kapsamı ve tutarlılık seçenekleri hizmete göre incelenir. "NoSQL veritabanları işlem desteklemez" veya "her zaman tutarsız okuma yapar" gibi genellemeler doğru bir seçim temeli oluşturmaz [17], [18], [20]–[22].

Tez teslim sisteminde öğrenci–tez–danışman–onay ilişkileri ve bunlar üzerindeki değişken sorgular, 5.2'de anlatılan ilişkisel veritabanı için güçlü bir gerekçedir. Buna karşılık öğrenci

kimliğiyle sıkça okunan bildirim tercihleri, erişim biçimi uygunsa yönetilen bir anahtar–değer veya belge hizmetinde tutulabilir. Örneğin `1045` anahtarıyla "e-posta bildirimi açık, SMS bildirimi kapalı" bilgisini bulabiliriz. Belge modelinde ise aynı öğrencinin bildirim tercihleri, `ogrenciNo: 1045`, `eposta: true`, `sms: false` alanlarını içeren tek bir kayıt olarak tutulabilir. NoSQL seçimi "veri çok büyüyecek" varsayımına değil; verinin yapısına, sorgulara, erişim sıklığına ve tutarlılık gereksinimine dayanmalıdır.

5.4. Önbellek ve Veri Gölü

Yönetilen ilişkisel ve NoSQL veritabanları, uygulamanın çalışan veri kaynakları olabilir. Önbellek ve veri gölü ise farklı ihtiyaçları karşılar: önbellek tekrarlanan erişimleri hızlandırır; veri gölü çeşitli kaynaklardan gelen verileri daha sonra analiz edebilmek için toplar.

Cache (önbellek), sık kullanılan bir verinin veya sorgu sonucunun geçici kopyasını uygulamaya hızlı erişilebilen bir yerde tutar. Örneğin tez teslim döneminde binlerce kullanıcı aynı "teslim koşulları" sayfasını açıyorsa, sayfa içeriği veya onu oluşturan sorgu sonucu önbellekte tutulabilir. Böylece her istekte ana veritabanının yeniden okunması gerekmez [23].

Önbellekteki bilgi eski kalabilir. Teslim koşulları değiştirildiğinde uygulama eski kopyayı kullanmaya devam ederse öğrencilere yanlış bilgi gösterilir. Bu nedenle önbellekteki kaydın ne zaman silineceği, güncelleneceği veya süresinin dolacağı belirlenmelidir. Önbellek, öğrenci başvuruları ve onaylar gibi kalıcı kayıtların tek doğru kaynağı olarak düşünülmemelidir [23].

Data lake (veri gölü), farklı kaynak ve biçimlerdeki verilerin analiz amacıyla bir araya getirildiği veri ortamıdır. Örneğin üniversite, yıllara göre tez teslim sayılarını incelemek için teslim kayıtlarının dışa aktarımlarını, kullanım kayıtlarını ve izin verilen diğer veri kümelerini bir veri gölünde toplayabilir. Bu veriler ilk toplandıklarında ham biçimlerini koruyabilir; analiz için gerekli düzenleme daha sonra yapılabilir [24].

Veri gölü kurmak, dosyaları tek bir depolama alanına yüklemekten ibaret değildir. Veri kümelerinin ne içerdiği, nereden geldiği, hangi döneme ait olduğu ve kimler tarafından kullanılabileceği anlaşılır olmalıdır. Tez teslim uygulaması günlük işlemlerini yürütürken ilişkisel veritabanını kullanmaya devam eder; veri gölü bu işlemleri doğrudan yönetmek veya veritabanının yedeği olmak için seçilmez [24].

Kısaca, önbellek "aynı bilgiye tekrar erişimi nasıl hızlandırırız?", veri gölü ise "farklı dönem ve kaynaklardaki verileri analiz için nasıl bir araya getiririz?" sorularına yanıt verir. Her ikisi de ancak somut bir gereksinim varsa mimariye eklenmelidir.

5.5. Veri Hizmeti Seçim Ölçütleri

Veri hizmeti seçerken önce saklanacak verinin biçimi, ardından uygulamanın bu veriye nasıl erişeceği belirlenir. "En hızlı" veya "en kolay ölçeklenen" hizmeti aramak tek başına

yeterli değildir. Bir hizmetin yararlı olması, gerekli sorguları ve güncellemeleri uygun biçimde desteklemesine bağlıdır [3], [17], [18].

Tablo 3, bu bölümde incelenen hizmetler için başlangıç niteliğinde bir seçim rehberidir.

Tablo 3. Veri türü ve erişim gereksinimine göre yönetilen veri hizmeti seçimi.

Gereksinim	**Öncelikle değerlendirilecek hizmet**	**Seçim gerekçesi**
Öğrenci, tez ve onay gibi ilişkili kayıtları sorgulama ve birlikte güncelleme	Yönetilen ilişkisel veritabanı	İlişkiler, sorgular ve veri bütünlüğü
Bir kaydı çoğunlukla bilinen kimliğiyle bulma	Uygun anahtar–değer NoSQL hizmeti	Önceden belirli erişim yolu
Bir varlığa ait alanları birlikte saklama ve belge olarak okuma	Uygun belge NoSQL hizmeti	Belge yapısına uygun erişim
Aynı, seyrek değişen bilgiyi çok sık okuma	Ana veri kaynağının yanında önbellek	Tekrarlanan okumaları azaltma
Farklı kaynak ve dönemlerden gelen verileri daha sonra analiz etme	Veri gölü	Analiz verilerini toplama ve düzenleme

Tablo tek başına karar vermez. Örneğin bir öğrenci kaydına kimlikle erişilebilmesi, o kaydın mutlaka NoSQL'de tutulması gerektiğini göstermez. Aynı kayıt, tez ve onay verileriyle ilişkiliyse ve bu ilişkiler sık sorgulanıyorsa ilişkisel veritabanı daha uygun olabilir. Tersine, belge modelinin seçilmesi de bütün alanlarda sınırsız ve aynı maliyetle sorgu yapılabileceği anlamına gelmez. Gerçekte yapılacak sorgular ve güncellemeler seçimden önce yazılmalıdır [17], [18].

Seçim sırasında şu beş soru yeterli bir başlangıç sağlar:

1. Veri hangi kayıtlardan oluşuyor ve aralarında hangi ilişkiler var?
2. Uygulama en sık hangi kayıtları, hangi anahtar veya koşullarla okuyacak?
3. Hangi kayıtların birlikte ve tutarlı biçimde güncellenmesi gerekiyor?
4. Okuma ve yazma yükü ne kadar; verinin kısa süre eski görünmesine izin verilebilir mi?
5. Yanlış silme veya güncellemede hangi duruma geri dönülmesi gerekiyor?

Tez teslim sistemi için makul bir ilk tasarım; tez PDF'lerini nesne depolamada, öğrenci–tez–onay kayıtlarını yönetilen ilişkisel veritabanında tutmaktır. Ölçülen bir tekrar eden okuma yükü varsa önbellek eklenebilir. Dönemler arası analiz gereksinimi doğarsa ilgili veri kümeleri veri gölünde düzenlenebilir. NoSQL ise ancak bildirim tercihleri gibi belirli bir veri kümesinin erişim biçimi bunu gerekçelendiriyorsa değerlendirilir. Böylece her hizmet, mimaride açıkça tanımlanmış bir görevi yerine getirir.

6. Kavramların Bulut Sağlayıcılarındaki Karşılıkları

Bu bölümdeki hizmet adları, önceki bölümlerde öğrenilen kavramları üç sağlayıcıda tanımak içindir. Aynı satırdaki hizmetler benzer bir ihtiyaca karşılık gelir; özellikleri bire bir aynı olmak zorunda değildir.

6.1. Depolama modelleri

Tablo 4. Üç depolama modeline ilişkin hizmet adı örnekleri. [25]–[27]

Depolama modeli	Amazon Web Services (AWS)	Microsoft Azure	Google Cloud
Blok depolama	Amazon EBS	Azure Managed Disks	Persistent Disk
Dosya depolama	Amazon EFS	Azure Files	Filestore
Nesne depolama	Amazon S3	Azure Blob Storage	Cloud Storage

6.2. Yönetilen veri hizmetleri

Tablo 5. Yönetilen veri hizmetlerine ilişkin örnek adlar. [28]–[30]

Gereksinim	AWS	Microsoft Azure	Google Cloud
İlişkisel veritabanı	Amazon RDS	Azure SQL Database	Cloud SQL
NoSQL veritabanı	Amazon DynamoDB	Azure Cosmos DB	Firestore
Önbellek	Amazon ElastiCache	Azure Managed Redis	Memorystore

Veri gölü, tek bir ürün türünden çok, verilerin saklanma ve kullanılma biçimini anlatır. Bu amaçla AWS'de Amazon S3, Azure'da Azure Data Lake Storage, Google Cloud'da Cloud Storage kullanılabilir. Bir çözümü seçerken hizmet adından önce verinin yapısı, erişim biçimi ve koruma gereksinimi belirlenmelidir.

7. Gerçek Dünya Kullanım Senaryosu: Üniversite Tez Teslim Sistemi

Bir üniversitenin tez teslim sisteminde öğrenciler tez dosyalarını yükler; danışmanlar ve yetkili personel dosyaları inceler. Sistem, dosyanın kendisini, teze ilişkin kayıtları ve sonradan yapılan değişiklikleri birlikte yönetmelidir. Bu senaryo, önceki bölümlerde ele alınan depolama ve veri hizmetlerinin hangi gereksinimler için seçildiğini gösterir.

7.1. Üniversite Tez Teslim Sisteminin Gereksinimleri

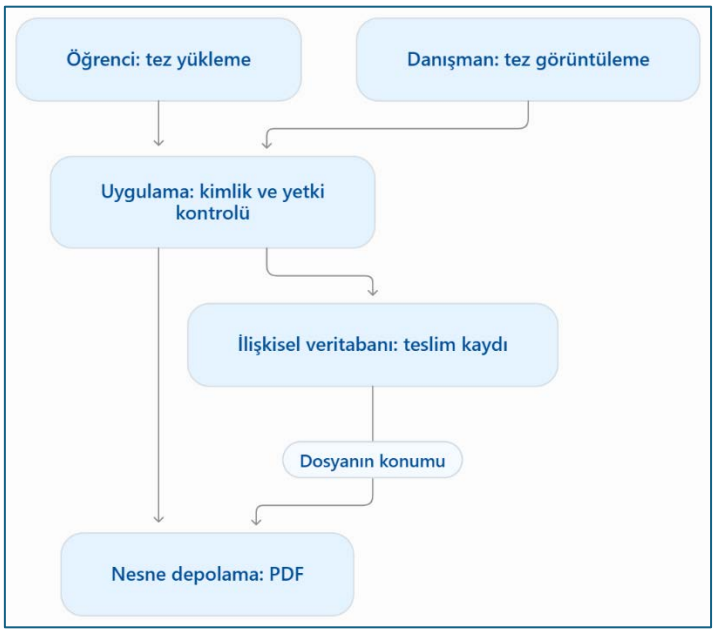
Her tez tesliminde iki farklı veri bulunur:

- **Tez dosyası:** Öğrencinin yüklediği PDF belgesi.
- **Teslim kaydı:** Öğrenci numarası, tez başlığı, danışman, teslim zamanı, dosyanın konumu ve değerlendirme durumu.

Öğrenci kendi teslimini görebilmeli; danışman yalnızca yetkili olduğu tezlere erişebilmelidir. Dosyanın yanlışlıkla silinmesi veya teslim kaydının hatalı güncellenmesi durumunda geri dönüş yolu bulunmalıdır. Dönem sonu gibi yoğun zamanlarda artan yükleme sayısı da kapasite planlamasında dikkate alınır.

7.2. Verinin Yerleştirilmesi ve Erişim Akışı

Tez PDF'leri, her dosyanın ayrı bir nesne olarak saklanabildiği nesne depolamada tutulur. Öğrenci, tez, danışman ve değerlendirme durumu arasındaki ilişkiler ise yönetilen ilişkisel veritabanında tutulur. Veritabanındaki teslim kaydı, ilgili PDF'nin nesne depolamadaki konumunu gösterir. Böylece bir tez arandığında önce kayıt bulunur, ardından yetki kontrolünden geçen kullanıcı dosyaya erişir.



Şekil 3. Tez dosyası ile teslim kaydının ayrı hizmetlerde tutulması ve yetkili erişim akışı.

Bu örnekte dosya ile teslim kaydının farklı yerlerde bulunması, ikisinin ilişkisiz olduğu anlamına gelmez. Uygulama, yükleme sırasında her iki işlemin sonucunu kontrol etmeli; dosyası yüklenemeyen bir tezi başarıyla teslim edilmiş gibi göstermemelidir.

7.3. Yaşam Döngüsü ve Geri Yükleme Kararları

Aktif değerlendirme döneminde tez PDF'lerine sık erişilir. Değerlendirme bittikten sonra erişim azalırsa dosyalar, üniversitenin saklama süresi ve geri alma hızı gereksinimine göre daha düşük maliyetli bir depolama katmanına geçirilebilir.

Geri dönüş için hazırlık, sistem kurulurken yapılır. Tez dosyalarının bulunduğu nesne depolama alanında sürümleme etkinleştirilir. Bundan sonra aynı dosya değiştirildiğinde veya silindiğinde hizmet, önceki sürümü belirlenen saklama kuralları çerçevesinde tutar. Yönetilen veritabanında ise otomatik yedekleme, saklama süresi ve destekleniyorsa belirli bir zamana geri yükleme ayarlanır. Böylece hizmet, kurtarma için gereken yedekleri ve

değişiklik kayıtlarını zaman içinde oluşturur. Ayarlar hata olduktan sonra açılırsa geçmişte hiç saklanmamış bir sürüm veya geri yükleme noktası oluşturulamaz. [31]–[34]

Örneğin öğrenci 14.00'te doğru PDF'yi yüklemiş, 14.20'de yanlış bir PDF ile değiştirmiş olsun. Yetkili kişi bulutun **nesne depolama yönetim ekranında** ilgili dosyayı açar, *sürümler* listesinden 14.00'teki dosyayı seçer ve onu yeniden geçerli sürüm hâline getirir. Dosya bu listede görünüyorsa kaynak, uygulamanın bilgisayarındaki bir kopya değil, nesne depolama hizmetinin sakladığı önceki sürümdür. [31]–[33]

Başka bir durumda PDF doğru kalmış, ancak 14.20'de veritabanındaki teslim durumu yanlış güncellenmiş olsun. Yetkili kişi bu kez yönetilen veritabanının yedekleme ve geri yükleme ekranına gider; saklama süresi içindeki, hatadan önceki bir zamanı (örneğin 14.19) seçer. Hizmetin sunduğu yönteme göre o zamandaki veritabanı durumu ayrı bir veritabanı veya örnek olarak oluşturulur. Ardından doğru teslim kaydı kontrol edilerek geri alınır. Tüm canlı veritabanını doğrudan eski hâline çevirmek, 14.19'dan sonra yapılmış doğru teslimleri de kaybettirebileceği için uygun değildir. [34]–[36]

Son olarak uygulamadaki teslim kaydı ile açılan PDF karşılaştırılır: öğrenci, tez başlığı, dosya ve teslim durumu aynı işleme ait olmalıdır. Böylece geri yüklemenin yalnızca tamamlanmış olması değil, doğru veriyi geri getirmiş olması da doğrulanır.

8. Bölüm Özeti

Bulutta depolama seçimi; verinin yapısına, erişim biçimine, kapasite ve performans gereksinimine dayanır. Maliyet değerlendirmesinde saklanan veri miktarının yanında erişim sıklığı, işlem sayısı ve veri aktarımı da dikkate alınır.

- Blok depolama, uygulamanın disk gibi kullanacağı depolama birimleri sunar.
- Dosya depolama, dosya ve klasör yapısıyla erişim sağlar; ortak dosya kullanımına uygundur.
- Nesne depolama, dosyaları içerikleri ve üst verileriyle ayrı nesneler olarak saklar; tez PDF'leri, görseller ve benzeri içerikler için uygundur.

Verinin erişim sıklığı zaman içinde değişebilir. Yaşam döngüsü kuralları, veriyi uygun depolama katmanlarına taşıyarak maliyetin yönetilmesine yardımcı olur. Katman seçilirken geri alma süresi ve erişim maliyeti de değerlendirilir.

Sürümleme, çoğaltma ve yedekleme farklı amaçlara hizmet eder. Sürümleme önceki nesne sürümlerini korur; çoğaltma verinin başka kopyalarını tutarak erişilebilirliği destekler; yedekleme ise veri kaybı veya hatalı değişikliklerden sonra geri yükleme için kullanılır. Geri dönüşün mümkün olması, gerekli koruma ayarlarının önceden yapılmasına ve ilgili sürüm ya da yedeğin hâlâ saklanmasına bağlıdır.

Yönetilen veri hizmetlerinde sağlayıcı, seçilen hizmetin kapsamına göre altyapı ve işletim görevlerinin bir kısmını üstlenir. Kullanıcı ise veri modelinden, erişim yetkilerinden ve koruma ayarlarının gereksinimlere uygunluğundan sorumludur. İlişkisel veritabanları ilişkili kayıtlar ve işlem bütünlüğü için; NoSQL veritabanları uygun veri modelleri ve erişim örüntüleri için değerlendirilir. Önbellek sık kullanılan verilere erişimi hızlandırır; veri gölü farklı yapıdaki verileri sonraki işleme ve analizler için bir arada saklar.

Tez teslim sistemi örneğinde PDF'lerin nesne depolamada, teslim kayıtlarının ilişkisel veritabanında tutulması bu seçimleri somutlaştırır. Bir hata sonrasında dosya sürümü veya veritabanı geri yükleme noktası seçilir; ardından dosya ile teslim kaydının tutarlı olduğu doğrulanır. AWS, Azure ve Google Cloud'daki hizmet adları değişse de bu kararların temelindeki kavramlar ortaktır.

Kaynakça

[1] "Storage - AWS Well-Architected Framework (2023-04-10)." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/2023-04-10/framework/perf-storage.html
[2] "Design an optimal storage strategy for your cloud workload | Cloud Architecture Center," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/architecture/storage-advisor
[3] nabilshams, "Choose a data storage technology - Azure Architecture Center." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/data-storage
[4] normesta, "Introduction to Azure Storage - Cloud storage on Azure." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/common/storage-introduction
[5] "About Cloud Storage objects," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/storage/docs/objects
[6] "Managing the lifecycle of objects - Amazon Simple Storage Service." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html
[7] normesta, "Azure Blob Storage lifecycle management policy structure - Azure Blob Storage." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/blobs/lifecycle-management-policy-structure
[8] "Object Lifecycle Management | Cloud Storage," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/storage/docs/lifecycle
[9] "Object Versioning | Cloud Storage," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/storage/docs/object-versioning
[10] "Replicating objects within and across Regions - Amazon Simple Storage Service." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html

[11] "Continuous backups and point-in-time recovery (PITR) - AWS Backup." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/aws-backup/latest/devguide/point-in-time-recovery.html
[12] kanshiG, "Continuous Backup with Point-in-Time Restore - Azure Cosmos DB." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/cosmos-db/continuous-backup-restore-introduction
[13] "Perform point-in-time recovery (PITR) | Cloud SQL for PostgreSQL," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/sql/docs/postgres/backup-recovery/pitr
[14] "What is Amazon Relational Database Service (Amazon RDS)? - Amazon Relational Database Service." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html
[15] WilliamDAssafMSFT, "What is the Azure SQL Database service? - Azure SQL Database." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/sql-database-paas-overview?view=azuresql
[16] "Cloud SQL documentation," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/sql/docs
[17] "Types of NoSQL databases - Choosing an AWS NoSQL Database." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/choosing-an-aws-nosql-database/types-of-nosql-databases.html
[18] claytonsiemens77, "Understand Data Models - Azure Architecture Center." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/understand-data-store-models
[19] "Data model | Firestore in Native mode," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/firestore/native/docs/data-model
[20] "What is Amazon DynamoDB? - Amazon DynamoDB." Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html
[21] markjbrown, "Partitioning and horizontal scaling - Azure Cosmos DB." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/cosmos-db/partitioning
[22] "Firestore in Native mode documentation," Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/firestore/native/docs
[23] claytonsiemens77, "Cache-Aside Pattern - Azure Architecture Center." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside
[24] PRASADA1207, "What Is a Data Lake? - Azure Architecture Center." Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/data-guide/scenarios/data-lake
[25] "PERF03-BP01 Use a purpose-built data store that best supports your data access and storage requirements - Performance Efficiency Pillar." Accessed: Sep.

29, 2026. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/performance-efficiency-pillar/perf_data_use_purpose_built_data_store.html
[26] normesta, “Introduction to Azure Storage - Cloud storage on Azure.” Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/common/storage-introduction
[27] “Storage,” Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/docs/storage
[28] “AWS Security Documentation.” Accessed: Sep. 29, 2026. [Online]. Available: https://docs.aws.amazon.com/security/?utm_source=chatgpt.com
[29] claytonsiemens77, “Prepare to Choose a Data Store in Azure - Azure Architecture Center.” Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/data-stores-getting-started
[30] “Memorystore,” Google Cloud Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://docs.cloud.google.com/memorystore/docs
[31] “Retaining multiple versions of objects with S3 Versioning - Amazon Simple Storage Service.” Accessed: Sep. 30, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html
[32] normesta, “Enable and manage blob versioning - Azure Storage.” Accessed: Sep. 30, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/blobs/versioning-enable
[33] “Use versioned objects | Cloud Storage,” Google Cloud Documentation. Accessed: Sep. 30, 2026. [Online]. Available: https://docs.cloud.google.com/storage/docs/using-versioned-objects
[34] “Introduction to backups - Amazon Relational Database Service.” Accessed: Sep. 30, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html
[35] WilliamDAssafMSFT, “Automatic, Geo-Redundant Backups - Azure SQL Database.” Accessed: Sep. 30, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-sql/database/automated-backups-overview?view=azuresql
[36] “Restore an instance overview | Cloud SQL for PostgreSQL | Google Cloud Documentation.” Accessed: Sep. 30, 2026. [Online]. Available: https://docs.cloud.google.com/sql/docs/postgres/backup-recovery/restore

Hafta 4 – Çalışma Soruları

Aşağıdaki sorular, bu haftanın temel kavramlarını açıklama ve farklı uygulama gereksinimlerine uygun çözümler geliştirme amacıyla hazırlanmıştır. Yanıtlarınızda seçimlerinizi gerekçelendiriniz. Senaryolarda birden fazla uygun çözüm bulunabilir; önerinizin hangi gereksinimi karşıladığını belirtiniz.

A. Depolama Gereksinimleri ve Modelleri

1. Blok, dosya ve nesne depolamayı uygulamaya sundukları yapı ve veriye erişim biçimi bakımından karşılaştırınız. Aynı PDF belgesinin dosya veya nesne depolamada saklanabilmesi, depolama seçiminin dosya uzantısına göre yapılamayacağını nasıl gösterir?

2. Kapasite, gecikme, aktarım hızı ve saniyedeki giriş/çıkış işlemi sayısı (IOPS) hangi gereksinimleri ölçer? Büyük video dosyalarını toplu olarak aktaran bir uygulama ile çok sayıda küçük ve rastgele okuma yapan bir uygulamada hangi performans ölçütleri öne çıkar? Yüksek aktarım hızı neden tek başına düşük gecikme anlamına gelmez?

3. Bir mimarlık bürosu, buluta taşıyacağı sistemde aşağıdaki gereksinimlere sahiptir:

    - Bir sanal makinenin işletim sistemi ve kurulu yazılımları için disk alanı gerekmektedir.
    - Mevcut çizim yazılımı, üç sunucunun aynı dosya ve dizin yollarına erişmesini beklemektedir.
    - Tamamlanmış proje görselleri, uygulama üzerinden bütün olarak yüklenip indirilmektedir; ortak dosya yolu gerekmemektedir.

    Her gereksinim için uygun depolama modelini seçerek gerekçelendiriniz. Bir blok birimin birden fazla sunucuya bağlanmasının neden tek başına güvenli ortak dosya erişimi sağlamadığını açıklayınız.

B. Yönetilen Veri Hizmetleri

4. Bir sanal makineye veritabanı yazılımı kurmak ile yönetilen veritabanı hizmeti kullanmak arasında işletim sorumlulukları bakımından ne fark vardır? Yönetilen hizmet kullanıldığında veri modeli, sorgular, erişim yetkileri ve geri yükleme hazırlığı konusunda uygulama ekibinin sorumlulukları neden devam eder?

5. Bir etkinlik bileti satış sisteminde müşteri, etkinlik, koltuk ve satış kayıtları bulunmaktadır. Bir satış tamamlandığında koltuğun durumu ile satış kaydının birbiriyle tutarlı olması ve aynı koltuğun iki müşteriye satılmaması gerekmektedir. Ayrıca müşterilerin satın aldığı biletler, etkinlik ve tarih bilgileriyle birlikte sorgulanmaktadır.

Bu gereksinimler yönetilen ilişkisel veritabanı seçimini nasıl destekler? Birbiriyle ilişkili kayıtların birlikte doğru güncellenmesinin önemini açıklayınız.

6. Bir akıllı tarım platformunda her cihazın ölçüm aralığı, alarm eşikleri ve etkin sensörleri saklanmaktadır. Cihazlar farklı ayar alanlarına sahip olabilir. Uygulama, ayarları çoğunlukla cihaz kimliğiyle bulup tek kayıt olarak okumaktadır.

   Anahtar–değer veya belge modeline sahip bir NoSQL hizmeti bu gereksinime neden uygun olabilir? Seçimden önce erişim ve güncelleme gereksinimleri hakkında hangi bilgileri toplarsınız? “NoSQL kullanılıyorsa veri modeli tasarlamak gerekmez” görüşünü değerlendiriniz.

7. Bir şehir içi ulaşım uygulamasında durak bilgileri çok sık okunmakta, ancak seyrek değişmektedir. Okuma yükünü azaltmak için önbellek kullanılmıştır. Bir durağın konumu ana veritabanında güncellendiği hâlde bazı kullanıcılara eski konum gösterilmektedir.

   Bu durumun nedenini açıklayınız ve önbelleğin güncelliğini korumak için bir yaklaşım öneriniz. Önbelleğin boşalması durumunda veriye nereden erişilmelidir?

8. Bir üretim işletmesi, farklı yıllara ait makine günlüklerini, sensör ölçümlerini ve bakım dosyalarını birlikte analiz etmek istemektedir. Günlük üretim işlemleri ise mevcut veritabanında yürütülmektedir.

   Veri gölü bu gereksinimde hangi görevi üstlenebilir? Verilerin kaynağı, dönemi, içeriği ve erişim yetkileri neden kaydedilmelidir? Veri gölünün günlük işlem veritabanı veya onun yedeği olarak kabul edilmesini değerlendiriniz.

C. Veri Yaşam Döngüsü ve Maliyet

9. Bir reklam ajansı, tamamladığı kampanyaların görsellerini ilk üç ay sık kullanmaktadır. Sonraki iki yıl erişim seyrekleşmekte, ardından görseller yalnızca arşiv amacıyla tutulmaktadır. Kurum içi saklama kuralı, görsellerin tamamlanma tarihinden itibaren toplam beş yıl korunmasını gerektirmektedir. İlk iki yıl içindeki erişimler hemen karşılanmalı; sonraki dönemde birkaç saatlik geri alma süresi kabul edilebilmektedir.

   Bu veriler için aşamalı bir yaşam döngüsü politikası öneriniz. Katman değiştirme ve silme koşullarını belirtiniz. Politikanın uygulanabilmesi için kampanyanın tamamlanma tarihi nasıl izlenmelidir?

10. Bir arşiv katmanının GB başına saklama ücreti daha düşüktür; ancak geri alma bedeli, asgari saklama süresi ve erken silme ücreti bulunmaktadır. Bu katmana geçirilen dosyalar beklenenden sık okunmaya başlanmıştır.

    Daha düşük saklama ücreti neden daha düşük toplam maliyet anlamına gelmeyebilir? Kararı yeniden değerlendirirken hangi maliyet ve erişim ölçütlerini incelersiniz?

D. Veri Koruma ve Geri Yükleme

11. Yaşam döngüsü yönetimi, sürümleme, çoğaltma ve yedekleme yöntemlerini temel amaçları bakımından karşılaştırınız. Aşağıdaki gereksinimlerin her biri için öncelikle hangi yöntemi değerlendirirsiniz?

    - Eski içerikleri daha düşük maliyetli bir depolama katmanına geçirmek.
    - Üzerine yanlış içerik yazılan bir nesnenin önceki hâline dönmek.
    - Bir konum kullanılamadığında başka konumdaki kopyaya erişmek.
    - Hatalı güncellenen veritabanını uygun bir geçmiş durumdan geri getirmek.

12. Bir yayınevinin dijital arşivinde, doğru kapak görselinin üzerine yanlış görsel yüklenmiştir. Yeni içerik ikinci konuma da çoğaltılmıştır. Hata öncesinden beri sürümleme etkindir ve doğru görselin önceki sürümü hâlâ saklanmaktadır.

    Çoğaltma neden bu hatadan geri dönüşü tek başına sağlamaz? Doğru görseli geri getirmek için hangi kaynağı kullanırsınız ve sonucu nasıl doğrularsınız? Sürümleme yalnızca hata sonrasında etkinleştirilseydi aynı geri dönüş mümkün olur muydu?

13. Bir araç kiralama sisteminde saat 11.40'ta yapılan hatalı toplu güncelleme, bazı kiralama kayıtlarının durumunu değiştirmiştir. Saat 11.40'tan sonra sisteme doğru yeni kiralamalar da eklenmiştir. Hata öncesinden beri belirli bir zamana geri dönüş özelliği etkindir ve ilgili geri dönüş noktaları saklanmaktadır.

    Hatalı kayıtları düzeltmek için bir geri yükleme yolu öneriniz. Tüm canlı veritabanını doğrudan 11.39'daki durumuna döndürmek hangi kayba yol açabilir? Hata öncesindeki durumu ayrı bir veritabanına geri yüklemek nasıl yardımcı olabilir?

14. Bir sigorta hasar sisteminde olay fotoğrafları nesne depolamada, hasar dosyaları ve fotoğrafların nesne anahtarları veritabanında tutulmaktadır. Bir hata sonrasında fotoğraflar ile veritabanı farklı zamanlardaki kopyalardan geri yüklenmiştir.

    Her iki geri yükleme işleminin başarılı olması neden uygulamanın doğru çalıştığını kanıtlamaz? Fotoğraflar ile hasar kayıtları arasındaki ilişkileri doğrulamak için en az üç kontrol öneriniz.

E. Bütünleşik Mimari Tasarımı

15. Bir çevrim içi el sanatları pazaryerinde aşağıdaki gereksinimler bulunmaktadır:

    - Uygulama bir sanal makinede çalışmaktadır.
    - Ürün görselleri bütün olarak yüklenip indirilmektedir.
    - Müşteri, sipariş ve ödeme kayıtlarının ilişkileri korunmalıdır.

- Mağaza tercihleri çoğunlukla mağaza kimliğiyle tek kayıt olarak okunmaktadır.
- Seyrek değişen kategori listesi çok sık sorgulanmaktadır.
- Geçmiş satış dışa aktarımları ve kullanım günlükleri dönemsel analizlerde birlikte kullanılacaktır.
- Ürün görsellerinin yanlışlıkla değiştirilmesine ve sipariş kayıtlarının hatalı güncellenmesine karşı geri dönüş gerekmektedir.

Bu sistem için basit bir mimari çiziniz. Sanal makine diski, ürün görselleri ve diğer veri kümeleri için uygun depolama veya veri hizmetlerini belirleyiniz. Önbelleğin ana veri kaynağıyla, ürün görsellerinin de ilgili kayıtlarla bağlantısını gösteriniz. Son olarak iki hata türü için geri dönüş yöntemlerini belirtiniz ve mağaza tercihleri için ayrı bir NoSQL hizmeti kullanmanın zorunlu olup olmadığını gerekçelendiriniz.
